

□ Maîtrise de l'Héritage en C++ - Académie des Ninjas

Bienvenue dans votre formation complète sur l'héritage en C++, où nous explorerons ce concept fondamental à travers l'univers passionnant de Naruto !

□ CONCEPT 1 : LES BASES DE L'HÉRITAGE

1. INTRODUCTION

L'héritage est l'un des piliers de la programmation orientée objet. Imaginez que vous entrez dans l'Académie des Ninjas de Konoha : tous les étudiants apprennent d'abord les techniques de base comme la transformation, la substitution et le clonage. Ensuite, selon leur clan d'origine, ils développent des spécialités particulières.

Après cette section, vous saurez :

- Comprendre pourquoi l'héritage évite la duplication de code
- Créer une classe de base et des classes dérivées
- Utiliser la syntaxe "class Enfant : public Parent"
- Identifier les relations "est-un" entre les classes

2. EXPLICATION FONDAMENTALE

Définition simple : L'héritage permet à une classe fille de récupérer automatiquement toutes les caractéristiques et comportements de sa classe mère, tout en pouvant ajouter ses propres spécialités.

Analogie familière : C'est exactement comme dans une famille de ninjas. Tous les membres du clan Uchiha héritent naturellement de certaines capacités de base, mais chacun peut développer ses propres techniques spéciales.

Difficultés typiques anticipées :

- Confusion entre "avoir" et "être" : Un Uchiha EST un Ninja (héritage), il n'A PAS un Ninja (composition)
- Oublier le mot-clé "public" dans la déclaration d'héritage
- Ne pas comprendre que la classe fille a TOUTES les propriétés de la classe mère

3. ILLUSTRATION CONCRÈTE

Voici un exemple détaillé étape par étape :

```

// ÉTAPE 1 : Créer la classe de base - Ninja
class Ninja {
protected: // Accessible aux classes filles
    string nom;
    int chakra;

public:
    // Constructeur de base
    Ninja(string n, int c) : nom(n), chakra(c) {
        cout << nom << " entre à l'Académie des Ninjas" << endl;
    }

    // Techniques que TOUS les ninjas connaissent
    void transformation() {
        cout << nom << " se transforme" << endl;
    }

    void afficherStats() {
        cout << nom << " a " << chakra << " points de chakra" << endl;
    }
};

// ÉTAPE 2 : Créer une classe dérivée - Uchiha
class Uchiha : public Ninja { // "public" est CRUCIAL ici !
private:
    bool sharinganActif;

public:
    // ÉTAPE 3 : Le constructeur appelle le constructeur parent
    Uchiha(string n, int c) : Ninja(n, c), sharinganActif(false) {
        cout << nom << " rejoint le clan Uchiha" << endl;
    }

    // ÉTAPE 4 : Ajouter des capacités spéciales
    void activerSharingan() {
        sharinganActif = true;
        cout << nom << " active son Sharingan !" << endl;
    }
};

```

Décomposition du raisonnement :

1. La classe Ninja définit ce que TOUT ninja peut faire

2. La classe Uchiha hérite de Ninja avec " : public Ninja"
3. Un Uchiha peut utiliser toutes les méthodes de Ninja PLUS ses propres méthodes
4. Le constructeur Uchiha DOIT appeler le constructeur Ninja

Utilisation :

```
int main() {  
    Uchiha sasuke("Sasuke", 100);  
    // Sasuke peut utiliser les méthodes héritées :  
    sasuke.transformation(); // Vient de Ninja  
    sasuke.afficherStats();   // Vient de Ninja  
    // ET ses propres méthodes :  
    sasuke.activerSharingan(); // Propre à Uchiha  
}
```

4. VÉRIFICATION DE COMPRÉHENSION

Question 1 : Quelle syntaxe est correcte pour faire hériter la classe Hyuga de la classe Ninja ?

- A. `class Hyuga extends Ninja`
- B. `class Hyuga : public Ninja`
- C. `class Hyuga inherits Ninja`
- D. `class Hyuga -> Ninja`

Question 2 : Si un objet Uchiha appelle la méthode `transformation()`, que se passe-t-il ?

- A. Erreur de compilation - Uchiha n'a pas cette méthode
- B. La méthode `transformation()` de la classe Ninja s'exécute
- C. Il faut d'abord créer un objet Ninja
- D. La méthode ne fait rien

Question 3 : Dans l'exemple, pourquoi utilise-t-on "protected" pour nom et chakra dans la classe Ninja ?

- A. Pour que seule la classe Ninja puisse y accéder
- B. Pour que les classes filles puissent y accéder directement
- C. Pour cacher ces variables au programmeur
- D. C'est obligatoire en C++

5. RÉCAPITULATION

Points essentiels à retenir :

- L'héritage crée une relation "est-un" : Un Uchiha EST un Ninja
- **Syntaxe :** `class Fille : public Mere`
- La classe fille hérite automatiquement de TOUT ce qui est public ou protected de la classe mère

- Le constructeur de la classe fille DOIT appeler le constructeur de la classe mère
- "protected" permet aux classes filles d'accéder aux membres

Pourquoi c'est important : L'héritage évite la duplication de code et crée une hiérarchie logique. Au lieu de réécrire "transformation()" dans chaque classe de ninja, on l'écrit une fois dans Ninja et toutes les classes filles en héritent automatiquement !

□ CONCEPT 2 : LES MODIFICATEURS D'ACCÈS DANS L'HÉRITAGE

1. INTRODUCTION

Dans le monde des ninjas, certaines techniques sont publiques (connues de tous), d'autres sont des secrets de famille (transmis seulement aux descendants), et certaines sont des secrets personnels (gardés pour soi). En C++, les modificateurs d'accès fonctionnent exactement de la même manière !

Après cette section, vous saurez :

- Distinguer public, protected et private dans l'héritage
- Choisir le bon niveau d'accès pour chaque membre
- Comprendre pourquoi "public" est généralement utilisé pour l'héritage

2. EXPLICATION FONDAMENTALE

Définition simple : Les modificateurs d'accès contrôlent qui peut utiliser quoi dans votre code, comme les niveaux de sécurité dans un village ninja.

Analogie familière :

- **Public** : Techniques connues de tout le village (Transformation, Substitution)
- **Protected** : Secrets de famille transmis aux descendants (Techniques secrètes du clan)
- **Private** : Secrets personnels gardés pour soi (Journal intime d'un ninja)

Difficultés typiques anticipées :

- Confondre "protected" et "private"
- Ne pas comprendre pourquoi utiliser "public" dans l'héritage
- Oublier que "protected" est accessible aux classes filles mais pas au public

3. ILLUSTRATION CONCRÈTE

```

class Ninja {
public:          // Accessible partout
    void sePresenter() {
        cout << "Je suis " << nom << " du village de Konoha" << endl;
    }

protected:    // Accessible aux classes filles uniquement
    string nom;
    int chakra;
    string techniqueFamiliare; // Secret de famille

private:      // Accessible seulement dans cette classe
    string journalIntime;      // Secret personnel
    int missionsSecretes;

public:
    Ninja(string n, int c) : nom(n), chakra(c) {
        journalIntime = "Mes pensées secrètes...";
        missionsSecretes = 0;
    }
};

class Uchiha : public Ninja {
public:
    Uchiha(string n, int c) : Ninja(n, c) {
        // Je peux accéder aux membres protected :
        techniqueFamiliare = "Katon : Boule de Feu";
        cout << nom << " connaît " << techniqueFamiliare << endl;

        // ERREUR ! Je ne peux PAS accéder aux membres private :
        // journalIntime = "Nouveau journal"; // INTERDIT !
    }

    void utiliserSecretFamiliare() {
        cout << nom << " utilise : " << techniqueFamiliare << endl;
        // Ceci fonctionne car techniqueFamiliare est protected
    }
};

int main() {
    Uchiha sasuke("Sasuke", 100);

    // Public : accessible partout

```

```

    sasuke.sePresenter(); // OK

    // Protected : NOT accessible depuis main()
    // cout << sasuke.nom; // ERREUR !

    // Private : NOT accessible nulle part sauf dans Ninja
    // cout << sasuke.journalIntime; // ERREUR !
}

```

Tableau récapitulatif visuel :

Niveau d'accès	Classe mère	Classes filles	Reste du code
-----	-----	-----	-----
public	✓	✓	✓
protected	✓	✓	✗
private	✓	✗	✗

4. VÉRIFICATION DE COMPRÉHENSION

Question 1 : Dans la classe Uchiha qui hérite de Ninja, que peut-on faire avec un membre "protected" de Ninja ?

- A. Rien, il est inaccessible
- B. Le lire et le modifier dans les méthodes de Uchiha
- C. Seulement le lire, pas le modifier
- D. L'utiliser seulement dans le constructeur

Question 2 : Pourquoi utilise-t-on généralement "public" dans "class Uchiha : public Ninja" ?

- A. C'est obligatoire en C++
- B. Pour que Uchiha puisse accéder aux membres de Ninja
- C. Pour maintenir les niveaux d'accès originaux de Ninja
- D. Pour éviter les erreurs de compilation

Question 3 : Un objet Uchiha créé dans main() peut-il accéder directement à un membre protected de sa classe mère ?

- A. Oui, toujours
- B. Non, jamais
- C. Seulement s'il est déclaré public
- D. Seulement dans les méthodes de la classe Uchiha

5. RÉCAPITULATION

Points essentiels à retenir :

- **Public** : accessible partout (techniques communes)

- **Protected** : accessible dans la classe et ses filles (secrets de famille)
- **Private** : accessible seulement dans la classe (secrets personnels)
- L'héritage "public" préserve les niveaux d'accès originaux
- Protected est parfait pour partager des données avec les classes filles sans les exposer au public

Pourquoi c'est important : Un bon contrôle d'accès protège vos données et crée une interface claire. Comme dans un village ninja, chacun doit savoir ce qu'il a le droit de voir et d'utiliser !

□ CONCEPT 3 : LES CONSTRUCTEURS DANS L'HÉRITAGE

1. INTRODUCTION

Quand un jeune ninja rejoint un clan, il doit d'abord passer par l'Académie générale avant d'apprendre les techniques spéciales de son clan. En C++, c'est pareil : quand on crée un objet d'une classe fille, le constructeur de la classe mère s'exécute TOUJOURS en premier !

Après cette section, vous saurez :

- Comprendre l'ordre d'exécution des constructeurs
- Passer des paramètres au constructeur parent
- Gérer l'initialisation des classes avec héritage

2. EXPLICATION FONDAMENTALE

Définition simple : Dans l'héritage, les constructeurs s'appellent en cascade : d'abord la classe mère, puis la classe fille, comme une chaîne de formation ninja.

Analogie familière : C'est comme l'apprentissage d'un ninja :

1. D'abord, il apprend les bases à l'Académie (constructeur Ninja)
2. Ensuite, il apprend les spécialités de son clan (constructeur Uchiha)

Difficultés typiques anticipées :

- Oublier d'appeler explicitement le constructeur parent
- Confusion sur l'ordre d'exécution
- Ne pas comprendre la syntaxe avec les deux-points

3. ILLUSTRATION CONCRÈTE

```

class Ninja {
protected:
    string nom;
    int chakra;
    string village;

public:
    // Constructeur de base
    Ninja(string n, int c, string v) : nom(n), chakra(c), village(v) {
        cout << "ÉTAPE 1 : " << nom << " entre à l'Académie de " << village << endl;
        cout << "Formation de base terminée !" << endl;
    }

    ~Ninja() {
        cout << nom << " quitte l'Académie" << endl;
    }
};

class Uchiha : public Ninja {
private:
    int niveauSharingan;

public:
    // SYNTAXE IMPORTANTE : Ninja(n, c, v) appelle le constructeur parent
    Uchiha(string n, int c, int niveau)
        : Ninja(n, c, "Konoha"), niveauSharingan(niveau) {
        cout << "ÉTAPE 2 : " << nom << " rejoint le clan Uchiha" << endl;
        cout << "Niveau Sharingan : " << niveauSharingan << endl;
    }

    ~Uchiha() {
        cout << nom << " quitte le clan Uchiha" << endl;
    }
};

// Exemple avec une classe encore plus spécialisée
class UchihaElite : public Uchiha {
private:
    string mangekyo;

public:
    UchihaElite(string n, int c, string technique)
        : Uchiha(n, c, 3), mangekyo(technique) {

```

```

        cout << "ÉTAPE 3 : " << nom << " maîtrise le Mangekyou : " << mangekyo << endl;
    }

    ~UchihaElite() {
        cout << nom << " abandonne le Mangekyou" << endl;
    }
};

```

Démonstration de l'ordre d'exécution :

```

int main() {
    cout << "=== CRÉATION D'UN NINJA ELITE ===" << endl;
    UchihaElite itachi("Itachi", 150, "Amaterasu");

    cout << "\n=== FIN DU PROGRAMME ===" << endl;
    // Les destructeurs s'exécutent automatiquement dans l'ORDRE INVERSE
    return 0;
}

```

Sortie attendue :

```

=== CRÉATION D'UN NINJA ELITE ===
ÉTAPE 1 : Itachi entre à l'Académie de Konoha
Formation de base terminée !
ÉTAPE 2 : Itachi rejoint le clan Uchiha
Niveau Sharingan : 3
ÉTAPE 3 : Itachi maîtrise le Mangekyou : Amaterasu

=== FIN DU PROGRAMME ===
Itachi abandonne le Mangekyou
Itachi quitte le clan Uchiha
Itachi quitte l'Académie

```

4. VÉRIFICATION DE COMPRÉHENSION

Question 1 : Dans quelle ordre s'exécutent les constructeurs dans une hiérarchie Ninja → Uchiha → UchihaElite ?

- A. UchihaElite, puis Uchiha, puis Ninja
- B. Ninja, puis Uchiha, puis UchihaElite
- C. Peu importe l'ordre
- D. Uchiha, puis Ninja, puis UchihaElite

Question 2 : Que se passe-t-il si on oublie d'appeler explicitement le constructeur parent ?

- A. Erreur de compilation
- B. Le constructeur par défaut de la classe mère est appelé
- C. Le programme plante
- D. Rien, tout fonctionne normalement

Question 3 : Quelle est la syntaxe correcte pour appeler le constructeur parent avec des paramètres ?

- A. `Uchiha(string n) { Ninja(n, 100, "Konoha"); }`
- B. `Uchiha(string n) : Ninja(n, 100, "Konoha") { }`
- C. `Uchiha(string n) :: Ninja(n, 100, "Konoha") { }`
- D. `Uchiha(string n) -> Ninja(n, 100, "Konoha") { }`

5. RÉCAPITULATION

Points essentiels à retenir :

- L'ordre des constructeurs : Classe mère → Classe fille
- L'ordre des destructeurs : Classe fille → Classe mère (inverse !)
- Syntaxe : `ClasseFille(params) : ClasseMere(params) { }`
- Si on n'appelle pas explicitement le constructeur parent, le constructeur par défaut est utilisé

Pourquoi c'est important : Comprendre l'ordre des constructeurs évite les bugs d'initialisation. Comme un ninja doit maîtriser les bases avant les techniques avancées, votre code doit initialiser les parties fondamentales avant les spécialisations !

☐ CONCEPT 4 : LES FONCTIONS VIRTUELLES ET LE POLYMORPHISME

1. INTRODUCTION

Imaginez un concours de techniques spéciales à Konoha : chaque ninja présente SA technique signature, même s'ils portent tous le même bandeau de ninja. Le polymorphisme en C++ fonctionne exactement ainsi : selon le type réel de l'objet, la bonne méthode est appelée automatiquement !

Après cette section, vous saurez :

- Comprendre la différence entre fonctions normales et virtuelles
- Utiliser le polymorphisme pour traiter différents objets de manière uniforme
- Créer des fonctions virtuelles pures pour forcer l'implémentation

2. EXPLICATION FONDAMENTALE

Définition simple : Le polymorphisme permet à différents types d'objets de répondre différemment au même appel de méthode. Les fonctions virtuelles garantissent que la bonne version de la méthode est appelée selon le type réel de l'objet.

Analogie familière : C'est comme dire "Montre-moi ta technique signature !" à un groupe de ninjas :

- Naruto va faire un Rasengan
- Sasuke va faire un Chidori
- Neji va faire la Rotation Céleste Même commande, résultats différents selon le ninja !

Difficultés typiques anticipées :

- Confusion entre liaison statique et dynamique
- Oublier le mot-clé "virtual"
- Ne pas comprendre pourquoi on a besoin de pointeurs ou références

3. ILLUSTRATION CONCRÈTE

```

class Ninja {
protected:
    string nom;

public:
    Ninja(string n) : nom(n) {}

    // Méthode NON virtuelle - liaison statique
    void sePresenter() {
        cout << "Je suis un ninja générique" << endl;
    }

    // Méthode virtuelle - liaison dynamique
    virtual void techniqueSignature() {
        cout << nom << " utilise une technique de base" << endl;
    }

    // Fonction virtuelle pure - OBLIGE les classes filles à l'implémenter
    virtual void cri() = 0;

    // Destructeur virtuel - TRÈS IMPORTANT !
    virtual ~Ninja() {}
};

class Uzumaki : public Ninja {
public:
    Uzumaki(string n) : Ninja(n) {}

    void sePresenter() { // Redéfinition simple (pas virtuelle)
        cout << "Je suis " << nom << " du clan Uzumaki !" << endl;
    }

    void techniqueSignature() override { // "override" = redéfinition virtuelle
        cout << nom << " utilise RASENGAN !" << endl;
    }

    void cri() override { // OBLIGATOIRE car fonction pure dans Ninja
        cout << nom << " : Dattebayo !" << endl;
    }
};

class Uchiha : public Ninja {
public:

```

```

Uchiha(string n) : Ninja(n) {}

void sePresenter() {
    cout << "Je suis " << nom << " du clan Uchiha" << endl;
}

void techniqueSignature() override {
    cout << nom << " utilise CHIDORI !" << endl;
}

void cri() override {
    cout << nom << " : Hmph..." << endl;
}

};

```

Démonstration du polymorphisme :

```

int main() {
    // Création des ninjas
    Uzumaki naruto("Naruto");
    Uchiha sasuke("Sasuke");

    cout << "=== SANS POLYMORPHISME ===" << endl;
    naruto.sePresenter();    // Version Uzumaki
    sasuke.sePresenter();    // Version Uchiha

    cout << "\n=== AVEC POLYMORPHISME ===" << endl;
    // Tableau de pointeurs vers la classe de base
    Ninja* equipe[] = {&naruto, &sasuke};

    for(int i = 0; i < 2; i++) {
        // La méthode virtuelle appelle la BONNE version !
        equipe[i]->techniqueSignature(); // Rasengan puis Chidori
        equipe[i]->cri();                // Dattebayo puis Hmph

        // La méthode NON virtuelle appelle toujours la version de base
        equipe[i]->sePresenter();        // "ninja générique" 2 fois
    }

    return 0;
}

```

Exemple avancé avec fonction template :

```

template<typename TypeNinja>
void organiserTournoi(TypeNinja& ninja1, TypeNinja& ninja2) {
    cout << "\n=== DÉBUT DU COMBAT ===" << endl;
    ninja1.techniqueSignature();
    ninja2.techniqueSignature();
    cout << "=== FIN DU COMBAT ===" << endl;
}

// Usage :
Uzumaki naruto("Naruto");
Uchiha sasuke("Sasuke");
organiserTournoi(naruto, sasuke); // Fonctionne grâce au polymorphisme !

```

4. VÉRIFICATION DE COMPRÉHENSION

Question 1 : Que se passe-t-il quand on appelle une méthode virtuelle via un pointeur de classe de base ?

- A. La méthode de la classe de base s'exécute toujours
- B. La méthode de la classe réelle de l'objet s'exécute
- C. Erreur de compilation
- D. Le comportement est imprévisible

Question 2 : Quelle est la différence principale entre "virtual" et "= 0" ?

- A. Aucune différence
- B. "virtual" permet la redéfinition, "= 0" l'oblige
- C. "= 0" est plus rapide
- D. "virtual" est obsolète

Question 3 : Pourquoi le destructeur doit-il être virtuel dans une classe de base ?

- A. C'est obligatoire en C++
- B. Pour éviter les fuites mémoire lors de la destruction via pointeur de base
- C. Pour améliorer les performances
- D. Ce n'est pas nécessaire

5. RÉCAPITULATION

Points essentiels à retenir :

- **virtual** : permet la redéfinition, appelle la version de la classe réelle
- **= 0** : fonction virtuelle pure, oblige les classes filles à l'implémenter
- **override** : indique explicitement qu'on redéfinit une fonction virtuelle
- Le destructeur virtuel évite les fuites mémoire
- Le polymorphisme permet de traiter différents objets de manière uniforme

Pourquoi c'est important : Le polymorphisme est la magie de l'orienté objet ! Il permet d'écrire du code générique qui fonctionne avec différents types d'objets, comme organiser un tournoi qui marche avec n'importe quel type de ninja !

❑ QUIZ RÉCAPITULATIF - ACADÉMIE DES NINJAS

Question 1 : Quelle syntaxe permet à la classe Hyuga d'hériter de la classe Ninja ?

- A. `class Hyuga extends Ninja`
- B. `class Hyuga : public Ninja`
- C. `class Hyuga inherits Ninja`
- D. `class Hyuga -> Ninja`

Question 2 : Dans une hiérarchie Ninja → Uchiha → UchihaElite, quel est l'ordre d'exécution des constructeurs ?

- A. UchihaElite → Uchiha → Ninja
- B. Ninja → Uchiha → UchihaElite
- C. L'ordre dépend du compilateur
- D. Uchiha → Ninja → UchihaElite

Question 3 : Un membre "protected" dans la classe Ninja est accessible :

- A. Partout dans le programme
- B. Seulement dans la classe Ninja
- C. Dans Ninja et ses classes filles uniquement
- D. Nulle part

Question 4 : Quelle est la différence entre une fonction virtuelle et une fonction virtuelle pure ?

- A. Aucune différence
- B. Virtuelle = optionnelle, virtuelle pure = obligatoire à redéfinir
- C. Virtuelle pure est plus rapide
- D. On ne peut pas avoir les deux dans la même classe

Question 5 : Dans ce code, qu'affiche l'appel `ninja->techniqueSignature()` ?

```
Ninja* ninja = new Uchiha("Sasuke");  
ninja->techniqueSignature(); // techniqueSignature est virtuelle
```

- A. La version de Ninja
- B. La version de Uchiha
- C. Erreur de compilation
- D. Les deux versions

Question 6 : Pourquoi utilise-t-on l'héritage public généralement ?

- A. C'est obligatoire
- B. Pour préserver les niveaux d'accès de la classe mère
- C. Pour améliorer les performances
- D. Pour éviter les erreurs de syntaxe

Question 7 : Si on oublie d'appeler le constructeur parent explicitement :

- A. Erreur de compilation
- B. Le constructeur par défaut de la classe mère est appelé
- C. Le programme plante à l'exécution
- D. Rien ne se passe

Question 8 : Le mot-clé "override" sert à :

- A. Créer une nouvelle méthode
- B. Indiquer qu'on redéfinit une méthode virtuelle
- C. Rendre une méthode privée
- D. Optimiser le code

Question 9 : Dans quelle situation a-t-on une relation "est-un" ?

- A. Une voiture a un moteur
- B. Un Uchiha est un Ninja
- C. Une maison a des fenêtres
- D. Un étudiant a des livres

Question 10 : Pourquoi le destructeur doit-il être virtuel dans une classe de base ?

- A. C'est une règle de style
- B. Pour éviter les fuites mémoire avec le polymorphisme
- C. Pour améliorer les performances
- D. Ce n'est jamais nécessaire

☐ CORRECTIONS DÉTAILLÉES

Concept 1 - Corrections

Question 1 - Réponse : B La syntaxe correcte est `class Hyuga : public Ninja`. Le mot-clé "extends" existe en Java, "inherits" n'existe pas, et "->" est utilisé pour les pointeurs, pas l'héritage.

Question 2 - Réponse : B L'objet Uchiha hérite de toutes les méthodes publiques et protected de Ninja, donc `transformation()` s'exécute normalement. C'est tout l'intérêt de l'héritage !

Question 3 - Réponse : B "protected" permet aux classes filles d'accéder directement aux membres, contrairement à "private" qui les cache complètement. C'est parfait pour partager des données avec les descendants.

Concept 2 - Corrections

Question 1 - Réponse : B Les membres protected sont accessibles dans les classes filles, donc on peut les lire ET les modifier dans les méthodes de Uchiha.

Question 2 - Réponse : C L'héritage public préserve les niveaux d'accès originaux : public reste public, protected reste protected. C'est généralement ce qu'on veut.

Question 3 - Réponse : B Un objet créé dans main() ne peut jamais accéder directement aux membres protected, même de sa propre classe. Il faut passer par des méthodes publiques.

Concept 3 - Corrections

Question 1 - Réponse : B Les constructeurs s'exécutent toujours de la classe la plus générale vers la plus spécialisée :
Ninja → Uchiha → UchihaElite.

Question 2 - Réponse : B Si on n'appelle pas explicitement le constructeur parent, C++ appelle automatiquement le constructeur par défaut (sans paramètres) de la classe mère.

Question 3 - Réponse : B La syntaxe avec les deux-points (:) est la liste d'initialisation, c'est la façon correcte d'appeler le constructeur parent.

Concept 4 - Corrections

Question 1 - Réponse : B C'est la magie du polymorphisme ! Grâce au mot-clé "virtual", C++ appelle la méthode de la classe réelle de l'objet, pas celle de la classe du pointeur.

Question 2 - Réponse : B "virtual" permet de redéfinir mais n'oblige pas. "= 0" crée une fonction virtuelle pure qui OBLIGE toutes les classes filles à l'implémenter.

Question 3 - Réponse : B Si le destructeur n'est pas virtuel, détruire un objet via un pointeur de classe de base n'appellera que le destructeur de la base, causant des fuites mémoire.

Quiz Récapitulatif - Corrections

Question 1 - B : Syntaxe standard C++ pour l'héritage public.

Question 2 - B : Ordre logique : base → spécialisée → très spécialisée.

Question 3 - C : Protected = secret de famille, accessible aux descendants uniquement.

Question 4 - B : Pure = abstraite, oblige l'implémentation dans les classes filles.

Question 5 - B : Polymorphisme ! La version Uchiha s'exécute car l'objet est réellement un Uchiha.

Question 6 - B : Public préserve l'interface originale de la classe mère.

Question 7 - B : C++ est tolérant, il appelle le constructeur par défaut automatiquement.

Question 8 - B : Override = documentation explicite qu'on redéfinit une méthode virtuelle.

Question 9 - B : "est-un" = héritage, "a-un" = composition.

Question 10 - B : Essentiel pour éviter les fuites mémoire avec le polymorphisme.

APPLICATIONS PRATIQUES

Dans les jeux vidéo :

- Hiérarchie de personnages : Character → Warrior → Paladin
- Système d'objets : Item → Weapon → Sword

Dans les interfaces graphiques :

- Widget → Button → IconButton
- Gestion uniforme des événements grâce au polymorphisme

Dans les systèmes de fichiers :

- File → TextFile, BinaryFile, ImageFile
- Traitement uniforme avec des méthodes virtuelles comme open(), read(), write()

L'héritage est partout en programmation moderne ! Il structure le code, évite la duplication et permet des architectures élégantes et maintenables. Comme un bon ninja maîtrise d'abord les bases avant les techniques avancées, maîtriser l'héritage vous ouvre la voie vers la programmation orientée objet avancée !